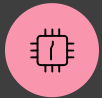


CUDA for Curious Kids

Compute
Unified
Device
Architecture

The CUDA Ecosystem



Hardware Architecture

Hardware specifications that allow producing CUDA-compatible GPU devices.

Compute capability = a set of features:

- qualitative: Tensor Cores, Ray Tracing Cores, TF32 support
- quantitative: SM memory size, #threads/SM, ...



Programming Model

Organizes computation into threads, blocks, grids, and kernels.

Links them them cores, warps, threads, SM's, ...



API Layers

Driver API (low-level C), Runtime API (C++-ish abstraction), Special APIs (i.e Tile API introduces in CUDA 13.1+ decreases thread-level management burden).



Toolkit & Libraries

Dev tools like nvcc compiler, cuda-gdb debugger, and core libraries: cuBLAS (linear algebra), cuDNN (deep learning), cuFFT (Fourier transforms), and more.

Driver API vs Runtime API

```
// Excerpt from
// https://github.com/NVIDIA/cuda-samples/blob/master/Samples/0_Introduction/vectorAdd/vectorAdd.cu
// https://github.com/NVIDIA/cuda-samples/blob/master/Samples/0_Introduction/vectorAddDrv/vectorAddDrv.cpp

// You are allowed to do this as a first cuda call
#include <cuda_runtime.h>
float *d_A = NULL;
err = cudaMalloc((void **)&d_A, size);

// -----
// But Driver API would require smth like
#include <cuda.h>
#include <helper_cuda_drvapi.h>
#include <helper_functions.h>
#include <builtin_types.h>

// Init CUDA and a device driver
checkCudaErrors(cuInit(0));
CUdevice cuDevice = findCudaDeviceDRV(argc, (const char **)&argv);
// Create context (~workspace for a program)
CUcontext cuContext;
CUctxCreateParams ctxCreateParams = {};
checkCudaErrors(cuCtxCreate(&cuContext, &ctxCreateParams, 0, cuDevice));

CUdeviceptr d_A;
checkCudaErrors(cuMemAlloc(&d_A, size));
```

Hardware Model: The GPU Architecture

CUDA works in a heterogeneous system—a host (CPU) and device (GPU) working together. Device code runs as kernels launched asynchronously relative to the host.



Graphics Processing Clusters (GPC)

Top-level partition containing multiple Streaming Multiprocessors



Streaming Multiprocessors (SM)

Core compute units containing CUDA cores, Tensor Cores, and hardware threads



Cores & Lanes

Tensor Cores, RT Cores, CUDA/SP cores, and hardware threads called lanes

SM Memory Types

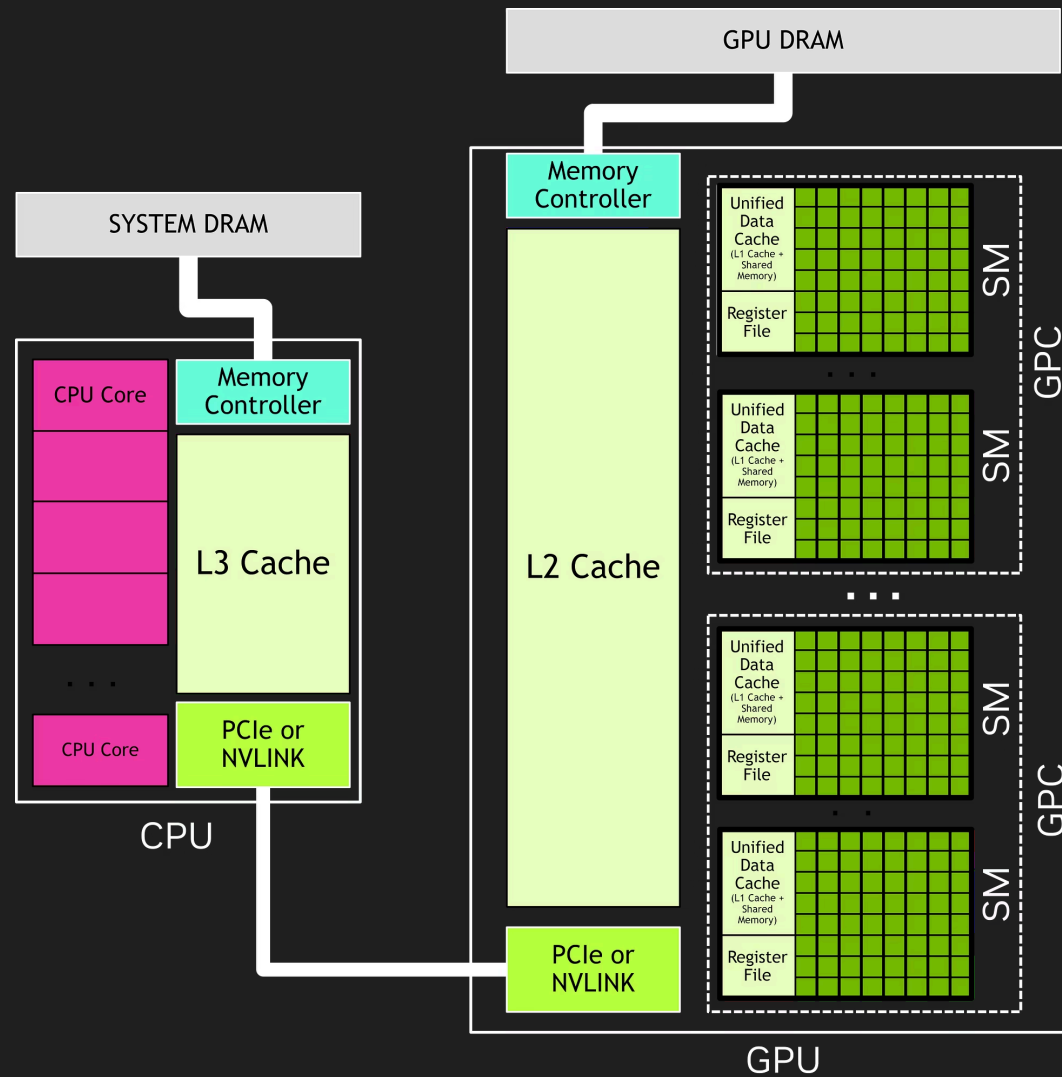
- Unified Data Cache (UDC): L1 cache + shared memory partitioned at runtime
- Register file: private storage for each thread

GPU Memory Hierarchy

- VRAM/Device Global Memory/High Band Memory (HBM)
- L2 cache (usually not managed by programmer)

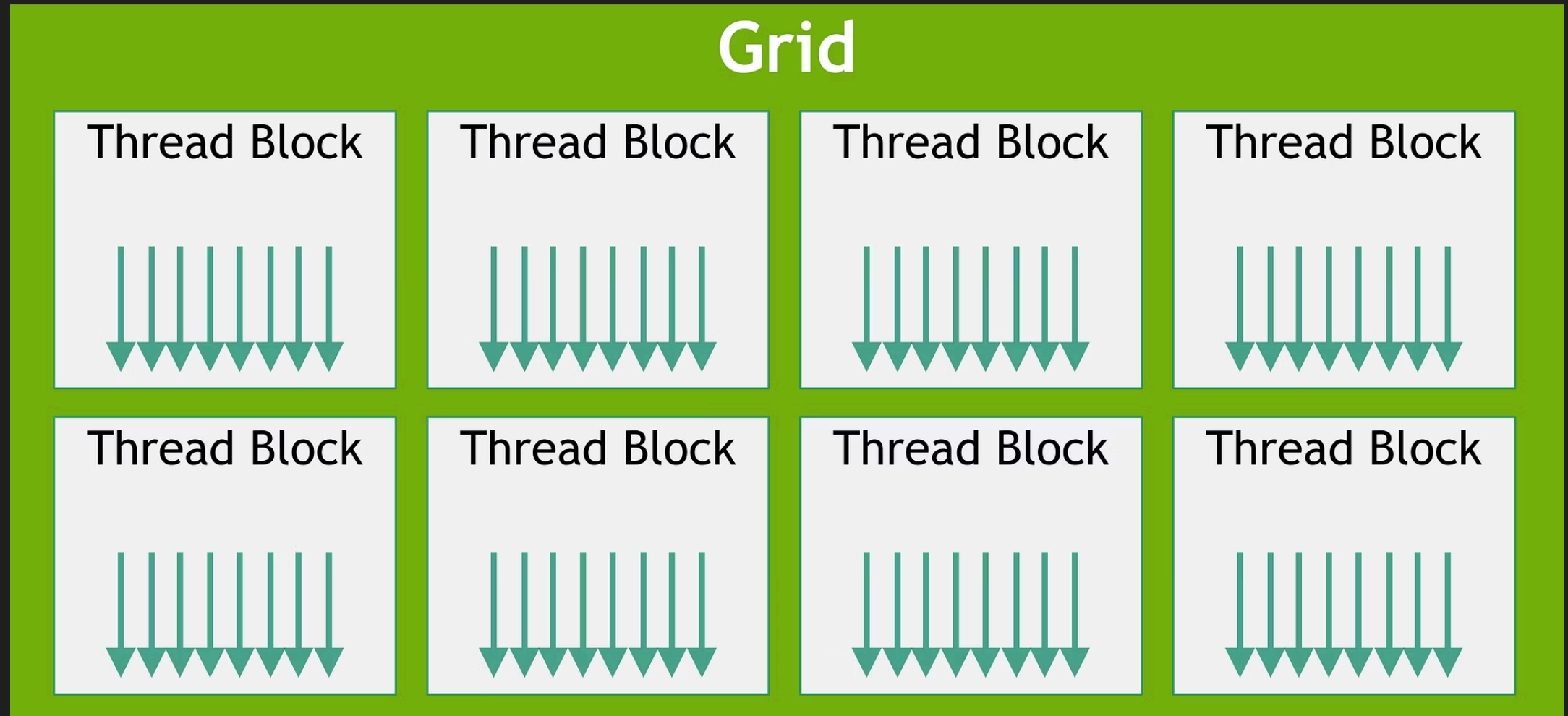
Interfaces

- PCIe/SXM (device to host)
- NVLINK (device to device)
- InfiniBand (network level)



Programming Model: Threads, Blocks & Grids

A kernel (device function) launches as massive amounts of threads grouped into thread blocks, which are themselves grouped into a Grid. All blocks have the same size and dimensions (1D, 2D, or 3D indexing).



Threads

Individual execution units. Each has unique ID within block and grid



Thread Blocks

Groups of threads executed on single SM. Can synchronize and share data



Grid

Collection of blocks launched by execution configuration

Example: Vector Addition

```
__global__ void vecAdd(const float *a, const float *b, float *c, int n)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}

// Host code
int main()
{
    int n = 1 << 20;          // number of elements
    int threadsPerBlock = 256; // block size (threads)
    int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;
    // allocate vectors a, b, c

    // Kernel launch: <<< grid, block >>> is the execution configuration
    vecAdd<<<blocksPerGrid, threadsPerBlock>>>(a, b, c, n);

    cudaDeviceSynchronize();
    return 0;
}

// General syntax is kernel<<<gridDim, blockDim, sharedMemBytes, stream>>>(args);
```

Programming Model: warps, SIMD and streams

Warps (32 threads)

Scheduled and executed in SIMT manner—same instruction, multiple threads. Behaves like 32-lane SIMD vector in lockstep.

Warp Execution

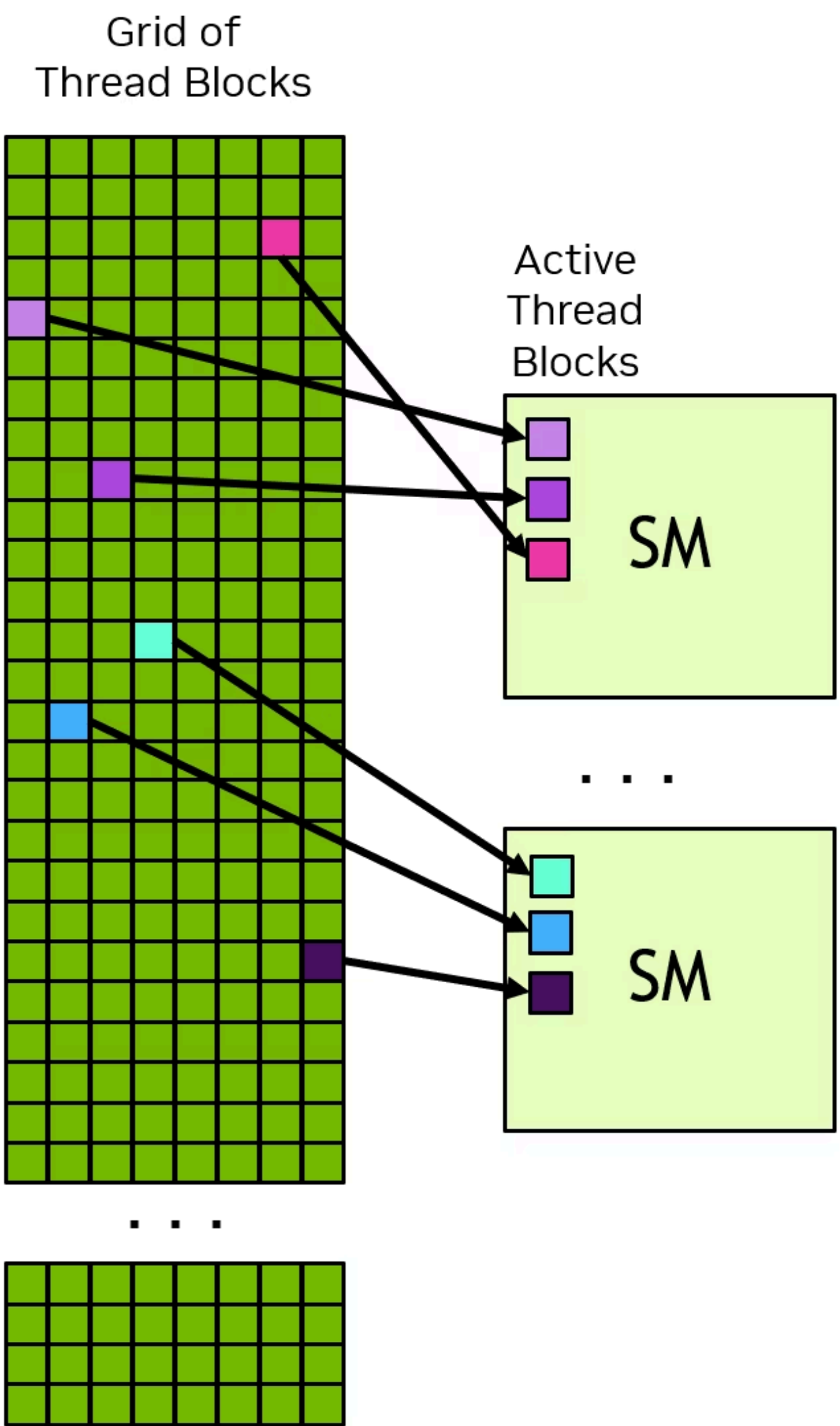
Unlike CPU SIMD, NVIDIA GPUs use many scalar execution units (CUDA cores) with hardware issuing operations over multiple cycles.

CUDA Streams

Instruction queues that schedule memory copies and kernels sequentially. Different streams execute asynchronously unless synchronized.

	Threads within the same warp	Same block, different warps	Different blocks
Run order	SIMT	may synchronize, but don't have to	no guarantees on scheduling: any order, any timing
Common memory	limited access (warp shuffle instructions) to the register of threads in the same warp (e.g. <code>__shfl_up(val, offset)</code>)	can use shared memory of an SM to exchange data	Generally should not have any cross-dependencies in execution

Programming Model: Threads, Blocks & Grids



Common Pitfalls & Performance Issues

Warp Lanes

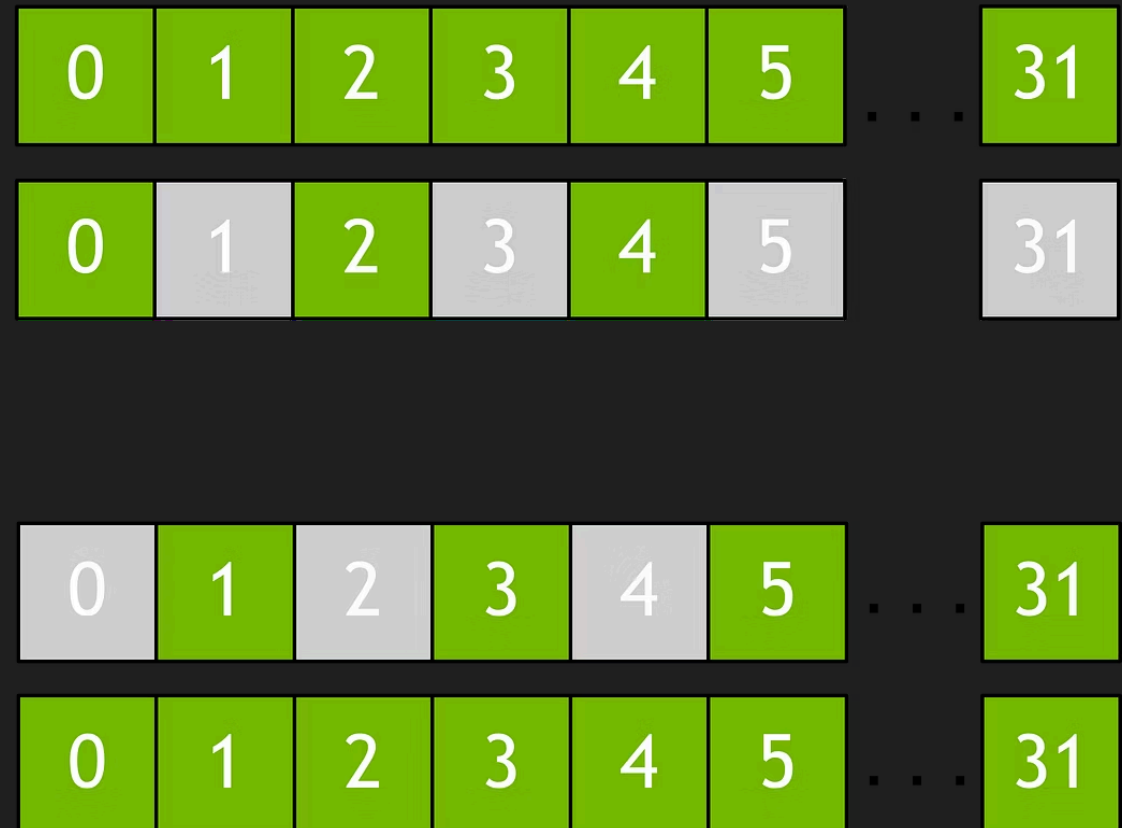
```
if(threadIdx.x%2 == 0)
```

```
{a = r(t); }
```

```
else
```

```
{a = q(t); }
```

```
y = f(a);
```



Register Pressure

Since a block must execute in a single SM, occupancy depends on available registers. If a block uses more than half of SM's registers, no other block from the same grid can execute concurrently, wasting compute power. Smaller blocks from other grids may still execute if resources allow.

Warp Divergence

When conditionals/branching cause threads in the same warp to follow different execution paths, performance suffers. Handled with masking—lanes not following an instruction are temporarily disabled. This is a **performance issue**, not correctness issue.

Resource scarcity at SM level

Similar to infamous register pressure, other SM resources (threads, shared memory, may also limit occupancy)

RTX A4000 Specifications (Compute Capability 8.6)

Resource / Limit	Value	Notes
Warp size	32 threads	Fixed across NVIDIA GPUs
Register file per SM	64K 32-bit	Shared by all resident warps
Max registers per thread	255	Compiler-enforced limit
Max threads per SM	1,536	48 warps per SM
Max warps per SM	48	Occupancy upper bound
Max thread blocks per SM	16	Architectural cap
Shared memory per SM	100 KB	Unified L1 + shared pool
Max shared memory per block	99 KB	Configurable
L1 cache per SM	Up to 128 KB	Ampere unified design
L2 cache (chip-wide)	~4–6 MB	GA104 ≈ 4 MB
Special function units (SFUs)	16 per SM	sin/cos/rsqrt
Tensor Cores per SM	4 (3rd-gen)	FP16/BF16/TF32
FP64 units per SM	2	1/32 rate vs FP32
FP32 CUDA cores per SM	128	Dual-issue capable
INT32 units per SM	64	Parallel with FP32
SM issue width	4 warp schedulers	1 warp/cycle each

📌 **Performance Note:** With 4 warp schedulers and 128 CUDA cores, only 4 warps can run instructions per cycle. However, up to 48 warps may be resident (code loaded, memory allocated). This is fine since memory operations have huge latencies (400-800 cycles for HBM), so most warps stall most of the time.

Tensor Core Performance

A4000 Tensor Core performance is usually reported with sparsity at BF16 data type. For Ampere architecture, Tensor Cores use same hardware paths for BF16, FP16, and TF32, so effective FLOPS = reported FLOPS / 2 $\approx$ 76.7 TFLOPS. FP32 is supported by CUDA cores only.

CUDA core FLOPS are computed based on available FMA operations (each comprised of 2 float ops):

```
48 SMs × 128 FP32 CUDA cores × 1.56 GHz boost clock × 2 ops = 19.2 TFLOPS
```

This is peak theoretical FLOPS. Real-world values may be 50-90% of that even for compute-bound scenarios.

Python Device Query

```
import torch
if torch.cuda.is_available():
    idx = torch.cuda.current_device()
    props = torch.cuda.get_device_properties(idx)
    print(f"Device index: {idx}")
    print(f"Name: {props.name}")
    print(f"Compute capability: {props.major}.{props.minor}")
    print(f"Total memory: {props.total_memory / 1024**3:.2f} GB")
    print(f"SM count: {props.multi_processor_count}")
    print(f"Max threads / SM: {props.max_threads_per_multi_processor}")
    print(f"Shared memory / block: {props.shared_memory_per_block / 1024:.1f} KB")
    print(f"Shared memory / SM: {props.shared_memory_per_multiprocessor / 1024:.1f} KB")
    print(f"Registers per SM: {props.regs_per_multiprocessor / 1024:.1f}K 32bit registers")
    print(f"Warp size: {props.warp_size}")
```

Floating Point Data Types

Format	Total Bits	Mantissa Bits	Exponent Bits	Approx. Range	Use Case
FP64 (double)	64	52	11	$\pm 1.8 \times 10^{308}$	HPC, scientific computing
FP32 (single)	32	23	8	$\pm 3.4 \times 10^{38}$	Standard DL operations
TF32 (tensor float)	19	10	8	$\pm 1.7 \times 10^{38}$	Matmuls, softmax, convolutions
FP16 (half)	16	10	5	$\pm 6.5 \times 10^4$	Accumulation in FP16/FP32
BF16 (bfloat16)	16	7	8	$\pm 3.4 \times 10^{38}$	Training and normalization
FP8 (E4M3)	8	3	4	± 240	LLM inference
FP8 (E5M2)	8	2	5	$\pm 57,344$	Large range variant

What's Next?

01

Torch Profiling

Simple tricks to speed things up (or slow them down dramatically)

03

Triton

CUDA kernels for the weak—simplified GPU programming

02

Memory vs Compute Bound

Understanding and mitigating performance bottlenecks

04

NCCL & Distributed Training

Beyond DDP—multi-GPU and multi-node communication

📄 **Attribution:** All images sourced from NVIDIA's official CUDA materials. Used for educational purposes only. NVIDIA and CUDA are trademarks of NVIDIA Corporation.